Marwadi University	Marwadi University Faculty of Technology Department of Information and Communication Technology	
Digital signal and Image process (01CT1513)	**Aim: Simulate linear convolution and circular convolution on discrete time signals.**	
Experiment - 2	**Date: 18/07/2026**	**Enrolment No:92400133125**

Aim : Simulate linear convolution and circular convolution on discrete time signals.

Github : https://github.com/mankumarbhalodiya/Digital-Signal-and-Image-Processing/tree/main/Experiment%202

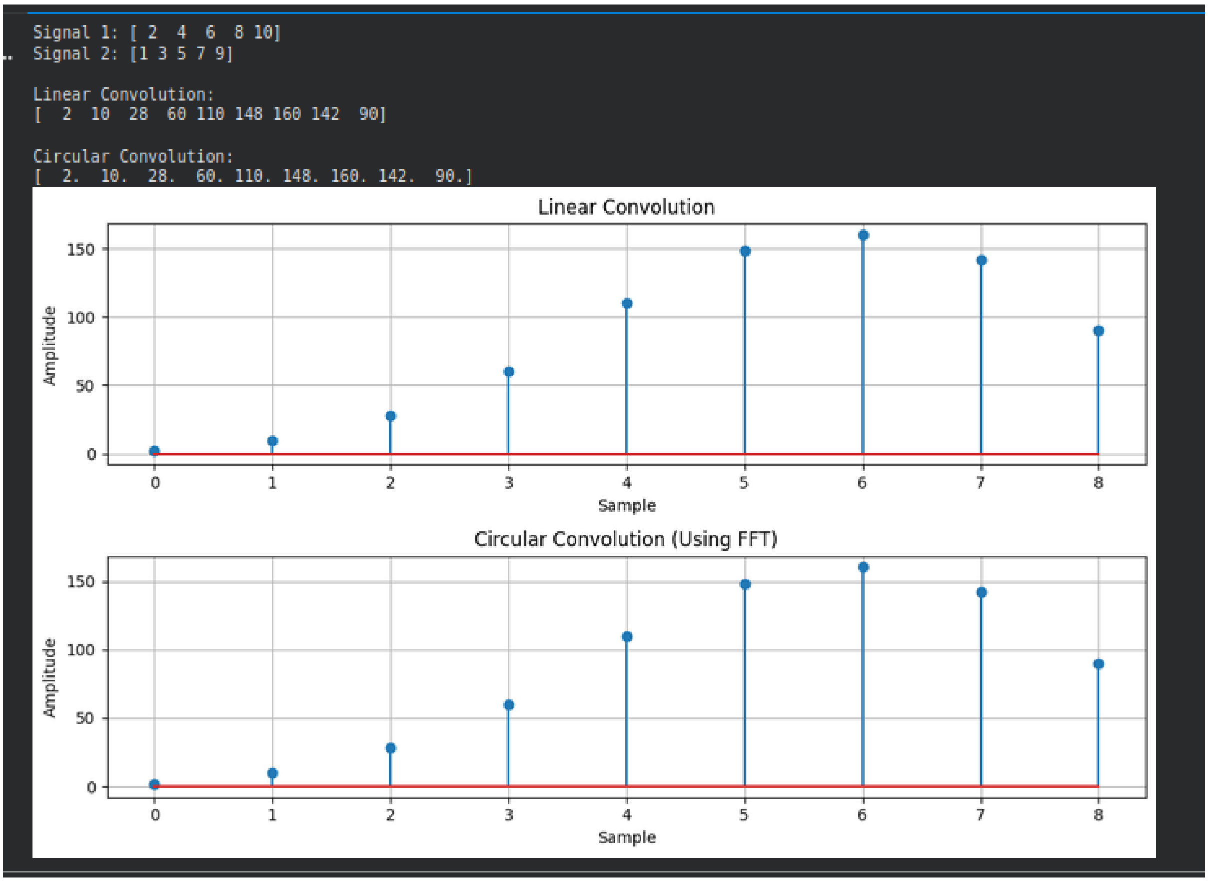
Code :

```
import numpy as np

import matplotlib.pyplot as plt

# Function to compute linear convolution

def linear_convolution(signal1, signal2):

    return np.convolve(signal1, signal2, mode='full')

# Function to compute circular convolution

def circular_convolution(signal1, signal2):

    fft_length = len(signal1) + len(signal2) - 1

    fft_signal1 = np.fft.fft(signal1, fft_length)

    fft_signal2 = np.fft.fft(signal2, fft_length)

    circular_conv = np.fft.ifft(fft_signal1 * fft_signal2)

    return circular_conv.real  # Take only the real part

# Define the discrete-time signals

signal1 = np.array([1, 2, 3, 4, 5])

signal2 = np.array([2, 4, 6, 8, 10])

# Compute the linear convolution

linear_conv = linear_convolution(signal1, signal2)

# Compute the circular convolution

circular_conv = circular_convolution(signal1, signal2)

 # Display the results

print("Signal 1:", signal1)
```

```
print("Signal 2:", signal2)

print("\nLinear Convolution:")

print(linear_conv)

print("\nCircular Convolution:")

print(circular_conv)

# Plot the results

plt.figure(figsize=(10, 6))

# Linear Convolution

plt.subplot(2, 1, 1)

plt.stem(range(len(linear_conv)), linear_conv)

plt.title("Linear Convolution")

plt.xlabel("Sample")

plt.ylabel("Amplitude")

plt.grid(True)

# Circular Convolution

plt.subplot(2, 1, 2)

plt.stem(range(len(circular_conv)), circular_conv)

plt.title("Circular Convolution (Using FFT)")

plt.xlabel("Sample")

plt.ylabel("Amplitude")

plt.grid(True)

plt.tight_layout()

plt.show()

# Optional: Save the results
```

```
# np.savetxt("linear_convolution.txt", linear_conv, delimiter=",")

# np.savetxt("circular_convolution.txt", circular_conv, delimiter=",")
```

OUTPUT :



Conclusion : This experiment helped me understand the difference between linear convolution and circular convolution. Linear convolution gives a longer output, while circular convolution gives an output of the same length as the input signals. Using Python made it easy to perform the calculations and visualize the results.

Marwadi University	Marwadi University Faculty of Technology Department of Information and Communication Technology	
Digital signal and Image process (01CT1513)	**Aim: Simulate linear convolution and circular convolution on discrete time signals.**	
Experiment - 2	**Date: 18/07/2026**	**Enrolment No:92400133125**

PAGE NO.:

Experiment - 2 Calculation

Q → Linear Circular

signal 1 = [2, 4, 6, 8, 10]

Signal 2 = [1, 3, 5, 7, 9]

	1	3	5	7	9
2	2	6	10	14	18
4	4	12	20	28	36
6	6	18	30	42	54
8	8	24	40	56	72
10	10	30	50	70	90

$y(0) = 2$

$y(1) = 4 + 6 = 10$

$y(2) = 6 + 12 + 10 = 28$

$y(3) = 8 + 18 + 20 + 14 = 60$

$y(4) = 10 + 24 + 30 + 28 + 18 = 110$

$y(5) = 30 + 40 + 42 + 36 = 148$

$y(6) = 50 + 56 + 54 = 160$

$y(7) = 70 + 72 = 142$

$y(8) = 90$

$x(n) * h(n) = \{2, 10, 28, 60, 110, 148, 160, 142, 90\}$

Marwadi University	Marwadi University Faculty of Technology Department of Information and Communication Technology	
Digital signal and Image process (01CT1513)	Aim: Simulate linear convolution and circular convolution on discrete time signals.	
Experiment - 2	Date: 18/07/2026	Enrolment No:92400133125

Circular Convolution

$$
\begin{bmatrix} 1 \\ 3 \\ 5 \\ 7 \\ 9 \\ 0 \\ 0 \\ 0 \\ 0 \end{bmatrix} \cdot
\begin{bmatrix}
2 & 0 & 0 & 0 & 0 & 10 & 8 & 6 & 4 \\
4 & 2 & 0 & 0 & 0 & 0 & 10 & 8 & 6 \\
6 & 4 & \cancel{6}2 & 0 & 0 & 0 & 0 & 10 & 8 \\
8 & 6 & 4 & 2 & 0 & 0 & 0 & 0 & 10 \\
10 & 8 & 6 & 4 & 2 & 0 & 0 & 0 & 0 \\
0 & 10 & 8 & 6 & 4 & 2 & 0 & 0 & 0 \\
0 & 0 & 10 & 8 & 6 & 4 & 2 & 0 & 0 \\
0 & 0 & 0 & 10 & 8 & 6 & 4 & 2 & 0 \\
0 & 0 & 0 & 0 & 10 & 8 & 6 & 4 & 2
\end{bmatrix}
$$

$$
= \begin{bmatrix}
2 \\
4+6 \\
6+12+10 \\
8+16+20+14 \\
10+24+30+28+18 \\
30+40+42+36 \\
50+56+54 \\
70+72 \\
90
\end{bmatrix}
= \begin{bmatrix} 2 \\ 10 \\ 28 \\ 60 \\ 110 \\ 148 \\ 160 \\ 142 \\ 90 \end{bmatrix}
$$